# Bags, Vectors & Transformers

Linguistic Analysis with spaCy · Bonus, Part 10

Denise J. Roth, PhD Candidate
Strategic Communication Group · Wageningen University & Research

# Beyond the Bag of Words

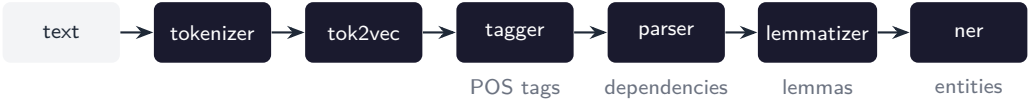
A bag of words knows *that* a speech mentions the government. Linguistic structure tells you more:

- is the government **acting** or **acted upon**?
- what does "we" **promise** to do?
- which **people, places, organizations** are named?
- has the **style** changed: sentence length, pronouns, nouns vs. verbs?

**spaCy**: the standard Python library for tokenizing, tagging, parsing and named entity recognition. Fast, with trained models for more than 20 languages, Dutch included.

**Running example:** all 60 US inaugural addresses, Washington 1789 to Trump 2025 (141,000 words), from NLTK.

# The Pipeline




```
nlp = spacy.load("en_core_web_sm")
doc = nlp(text)

for token in doc:
token.lemma_, token.pos_, token.dep_
doc.ents       # entities
doc.sents      # sentences
doc.noun_chunks # noun phrases
```

Models: `sm` fast · `md`/`lg` a bit better · `trf` transformer, best, slow on a CPU

| text | lemma | pos | dep | head |
|---|---|---|---|---|
| government | government | NOUN | nsubj | explain |
| did | do | AUX | aux | explain |
| n't | not | PART | neg | explain |
| explain | explain | VERB | ROOT | explain |
| rules | rule | NOUN | dobj | explain |

"The new government **didn't explain** the nitrogen rules to farmers in Gelderland . . . " The small model tags *Gelderland* as a PERSON: NER makes mistakes on names it rarely saw.

# Lemmas versus Stems

| word | Porter stem | spaCy lemma |
|---|---|---|
| studies | studi | study |
| policies | polici | policy |
| better | better | well |
| was | wa | be |
| organization | organ | organization |
| organized | organ | organize |
| general | gener | general |
| generous | gener | generous |

A **stem** (Day 1) chops endings by rule, without knowing the word. Often not a real word, and it lumps different meanings together.

A **lemma** is the dictionary form, using the part of speech: "saw" is *see* as a verb, *saw* as a noun. Better when you'll read the words, and for languages that inflect more, like Dutch.

Stemming is faster and fine for many bag-of-words tasks.

# Parts of Speech as Measurements

| period | words per sentence | "we" per 1,000 words | "I" |
|---|---|---|---|
| 1789–1849 | 43.4 | 17.5 | 22.1 |
| 1850–1899 | 31.5 | 18.2 | 14.3 |
| 1900–1949 | 23.5 | 44.5 | 7.8 |
| 1950–1999 | 20.6 | 59.2 | 9.7 |
| 2000–2025 | 18.9 | 63.0 | 10.1 |

Averages per speech. "We" = we, us, our, ours, ourselves; "I" = I, me, my, mine, myself; counted only when tagged as pronouns.

Sentences got **shorter**: written to be heard, not read. The president increasingly speaks **for the collective**, less about his own role.

spaCy's models were trained on modern English. Sentence boundaries and tags are less reliable for 18th-century prose. Trends this large survive that; small differences may not.

# Named Entities

`PERSON`, `GPE` (countries, cities), `LOC`, `ORG`, `NORP` (nationalities, political and religious groups), `DATE`, `MONEY`, ...

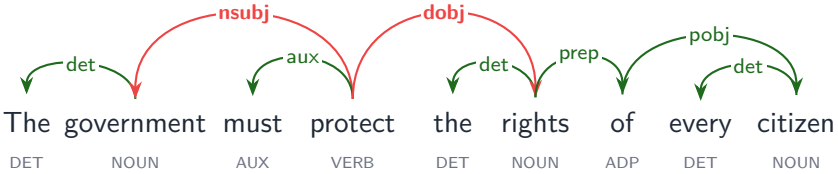
**Most frequent places, excluding the US**

| | |
|---|---|
| 1789–1899 | Texas, Washington, Great Britain, Louisiana, Spain |
| 1900–1999 | Philippines, Cuba, Vietnam, the Soviet Union |
| 2000–2025 | Detroit, Philadelphia, Panama, China |

**Check before counting.** The first "entity" in the corpus: *Fellow-Citizens*, tagged ORG. And one place has many spellings: *America*, *the United States*, *the United States of America*, and a partial *States*.

Check a sample per entity type, merge variants, and list what you exclude.

# Dependencies: Who Does What to Whom




Every word points to its **head**, with a relation label. From the tree: **subject – verb – object** triples, here `(government, protect, right)`.

4,552 triples from the 60 speeches. Aggregate them: what does *we* do, what happens *to* liberty?

Simple triples miss a lot: passives ("freedom must be defended"), clauses, coordinated subjects. Treat them as exploration and read the sentences behind the counts.

# Rule-Based Matching

The **Matcher** finds token sequences by their *attributes*: far more powerful than a regex on raw text.

```
matcher.add("WE_MODAL_VERB", [[
{"LOWER":  "we"},
{"LOWER":  {"IN":  ["will", "shall", "must", "can"]}},
{"POS":  "ADV", "OP":  "?"},  # optional adverb
{"POS":  "VERB"}]])
```

371 matches: "we will *always* defend", "we must act", ... The **PhraseMatcher** finds terms from a list (a Day 1 dictionary) quickly.

**Share of each modal among the matches**

| | can | must | shall | will |
|---|---|---|---|---|
| before 1945 | .24 | .42 | .23 | .11 |
| 1945 and later | .18 | .23 | .11 | .48 |

"We will" takes over. But part of this is a change in English as a whole: compare with a reference corpus before interpreting a trend.

# Dutch, and Working at Scale

## Same code, Dutch model

```
nlp_nl = spacy.load("nl_core_news_sm")
# reden -> rijden, trekkers -> trekker,
# stikstofregels -> stikstofregel
```

Lemmas matter more for Dutch (more inflection, separable verbs like *ik bel je op*). The Dutch model uses Universal Dependencies labels (`obj`, `obl`, `nmod`).

## Millions of documents

- `nlp.pipe(texts, batch_size=...)`, not a loop
- switch off what you don't need: `disable=["parser", "ner"]`
- `n_process=4` on several cores
- save parsed docs with `DocBin`

| 60 speeches, one CPU | time |
|---|---|
| full pipeline | 13 s |
| without parser and NER | 6 s |

# In the Notebook

- The pipeline, and the attributes of every token
- Tokenization, lemmas and stems compared
- Sentence length and “we” vs. “I” in 60 inaugural addresses
- Named entities, and the places presidents mention per century
- Dependency trees and subject–verb–object triples
- The Matcher and PhraseMatcher; Dutch; processing at scale

`bonus/notebooks/10_linguistic_analysis_with_spacy_exercises.ipynb` · 5 exercises · no GPU needed